Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Experiencia de Aprendizaje (EA4) - Proyecto de Aula (Fase 4)

Implementación de la estructura completa de una base de datos (DDL) en tres (3) SGBDs diferentes

- **Tarea en Equipo**
- **Peso: 15%**
- **Práctica. Caso de Estudio**
- **Modelo Físico - Diccionario de Datos en DBMS - PostgreSQL, MySQL, MS SQL Server**
- **Datos Estructurados y Semi Estructurados**

MIEMBROS DEL EQUIPO:

- Equipo E: Juan Camilo González Hoyos

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto de red social pascualina, ya logró mejorar los modelos conceptual y lógico; y en base al resultado, construyó un modelo físico en un SGBD en específico..

En este contexto, el reto consiste en tomar el modelo lógico y físico (en un SGBD específico) previamente construido y completamente funcional, e implementarlo en otros dos (2) Sistemas de Gestión de Bases de Datos (SGBD), aplicando rigurosamente las reglas del Lenguaje de Definición de Datos (DDL).

Es importante la identificación correcta de los tipos de datos de los SGBDs en los que implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico en los tres (3) SGBDs requeridos, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos genéricos en los tipos de datos del SGBD.

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo
- Justifiquen mejoras
- Transformen correctamente el modelo lógico en físico
- Implementen la base de datos en tres SGBD distintos:
  - PostgreSQL
  - MySQL
  - Microsoft SQL Server

3. Instrucciones de realización de la tarea

Experimentar el SGBD establecido, de acuerdo con las necesidades de la red social. Luego, utilizando una herramienta gráfica, como pgAdmin4, MySQL Workbench y MS SQL Server Studio, se enfocarán en la correcta implementación del modelo, prestando especial atención a la identificación de tablas, campos, los tipos y tamaños de datos, y, fundamentalmente, la definición de las claves primarias y las relaciones foráneas que dan coherencia al sistema. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final.

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones.
- Un cuadro comparativo de Tipos de Datos de los diferentes SGBDs
- Inventario de Tablas (resultado de la tarea anterior)
- Diccionario de Datos Físico para cada uno de los 3 SGBDs
- El conjunto de scripts con los modelos físicos (DDL) de los diferentes SGBDs.
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN** -> **Mostrar ejecución de scripts de creación y modificación.**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DDL y la implementación de una base de datos en tres (3) SGBD diferentes a partir del mismo Diccionario Lógico, habilidades fundamentales para cualquier desarrollador de software. Adicionalmente, tendrá la oportunidad de poder comparar la implementación en diferentes ambientes y la utilización de diversos tipos de datos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura: **Carpeta raíz: Tarea4. En esta carpeta estarán dos carpetas con los entregables: Informe, Resultados y Video.**

Carpetas	Contenido requerido	Observaciones
/tarea4	Todas las subcarpetas y archivos de la tarea. Nota: **USO OBLIGATORIO DE TODAS LAS PLANTILLAS SUMINISTRADAS**	No aplica
tarea4/Informe	Informe detallado	.pdf
tarea4/Diccionarios	Diccionario de Datos Físico (plantilla suministrada) ● Tabla comparativa de Tipos de Dato ● Inventario de Tablas ● Diccionarios de Datos	.xlsx
tarea4/Scripts	Script de creación de la base de datos para PostgreSQL Script de creación de la base de datos para MySQL Script de creación de la base de datos para MS SQL Server	.sql
	Script de modifcación de la base de datos PostgreSQL para incluir datos estructurados y semi estructurados	
tarea4/Video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales.	Enlace a YouTube

Evidencias de aprendizaje evaluadas

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: ● Identificación de los tipos de datos de los diferentes SGBD ● Descripción de cada tabla implementada con: nombre de la tabla, listado de campos con su tipo de dato y tamaño (si aplica), justificación de la elección de tipos de datos para campos clave (por ejemplo, por qué eligieron VARCHAR en lugar de TEXT para un campo específico, o INT en lugar de BIGINT), identificación de las claves primarias (PK yFK) y justificación de su elección. ● Descripción de cómo se implementaron las relaciones entre tablas. Para cada relación incluir: tablas involucradas (tabla padre, tabla hija y(o tabla pivote), campos que conforman la clave foránea, especificación de las acciones referenciales (ON DELETE, ON UPDATE, ON CASCADE) y su justificación. ● El informe debe referenciar los scripts DDL y explicar su estructura. ● Utilización de datos Semi Estructurados (JSON y/o JSONB) ● Conclusiones individuales y grupales individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso. ● Participación en Foros de Tareas	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.
Tipos de dato SQL y NoSQL para ***big data e IoT*** con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT Ejemplo: campo JSON para las coordenadas geográficas u optimización del tamaño del dato para captar señales de sensores en IoT.	Comprensión y utilización de tipos de datos que representen una relación con soluciones ***big data e IoT.***
Modelo físico con: ● Inventario completo de tablas de la base de datos. ● Tablas con nombres según las reglas y la nomenclatura. ● Campos correctamente identificados con tamaño y tipo asociado. ● Relaciones adecuadas al caso (claves primarias y foráneas). ● Uso correcto las reglas, nomenclatura y estándares para BD físicas. ● Utilización de datos tipo JSON o JSONB en alguna Tabla PostgreSQL	Comprensión del modelo físico
Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

Analicen los criterios de evaluación y verifiquen su cumplimiento..

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

1.- Tabla Comparativa de tipos de Datos

La tabla compara los tipos de datos genéricos del Anexo A con su implementación en los tres SGBD usados en la tarea: PostgreSQL 16, MySQL 8.0 y Microsoft SQL Server 2022. La tabla completa, con dominio y tamaño en bytes de cada tipo, está en la pestaña "Tipos de Dato" de la hoja de cálculo. El criterio general fue usar en los tres motores el tipo que conserva la misma regla de negocio, aunque el nombre cambie: un BOOLEAN de PostgreSQL es un BIT en SQL Server, y un JSONB de PostgreSQL es un JSON en MySQL y un NVARCHAR(MAX) validado con ISJSON en SQL Server.

Genérico	PostgreSQL 16	MySQL 8.0	SQL Server 2022	Observación
AUTOINCREMENTABLE	SMALLINT/INTEGER/BIGINT GENERATED ALWAYS AS IDENTITY	SMALLINT/INT/BIGINT AUTO_INCREMENT	SMALLINT/INT/BIGINT IDENTITY(1,1)	PK sustitutas. IDENTITY es el estándar SQL; AUTO_INCREMENT es propio de MySQL.
UUID	UUID (16 bytes)	BINARY(16) o CHAR(36)	UNIQUEIDENTIFIER (16 bytes)	No se usa en el modelo: las PK enteras (4 bytes) son suficientes. MySQL no tiene tipo UUID nativo.
ENTERO	SMALLINT / INTEGER / BIGINT	SMALLINT / INT / BIGINT	SMALLINT / INT / BIGINT	Iguales en los tres: 2, 4 y 8 bytes. SQL Server y MySQL además tienen TINYINT (1 byte).
DECIMAL	NUMERIC(p,s)	DECIMAL(p,s)	DECIMAL(p,s)	Exacto; para dinero (precios y totales en pesos). NUMERIC y DECIMAL son sinónimos.
LÓGICO	BOOLEAN	BOOLEAN (= TINYINT(1))	BIT	Atributos de dos valores. MySQL lo guarda como entero 0/1; SQL Server usa BIT 0/1.
TEXTO	TEXT	TEXT	NVARCHAR(MAX)	Contenido libre. En SQL Server TEXT está obsoleto.
CARACTER / TEXTO_CORTO	VARCHAR(n)	VARCHAR(n) (utf8mb4)	NVARCHAR(n)	Con límite de negocio. NVARCHAR guarda Unicode (tildes, ñ) a 2 bytes por carácter.
FECHA	DATE (4 bytes)	DATE (3 bytes)	DATE (3 bytes)	Sin hora: fecha_nacimiento, fecha_creacion.
FECHA_HORA	TIMESTAMP (8 bytes)	DATETIME (5 bytes)	DATETIME2(0) (6 bytes)	En MySQL TIMESTAMP llega solo a 2038; en SQL Server TIMESTAMP es rowversion (no es una fecha).
IMAGEN	VARCHAR(255) (URL) / BYTEA	VARCHAR(255) (URL) / LONGBLOB	NVARCHAR(255) (URL) / VARBINARY(MAX)	Se guarda la URL; el archivo va en almacenamiento externo.
AUDIO	VARCHAR(255) (URL) / BYTEA	VARCHAR(255) (URL) / LONGBLOB	NVARCHAR(255) (URL) / VARBINARY(MAX)	Mismo criterio que IMAGEN (no usado en el modelo).
URL	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	url_multimedia, url_fotografia.
ENUM	VARCHAR(n) + CHECK (IN ...)	VARCHAR(n) + CHECK (o ENUM(...))	NVARCHAR(n) + CHECK (IN ...)	Estados de 3 o más valores. Se usa CHECK en los tres para tener la misma regla.
BINARIO	BYTEA	VARBINARY / LONGBLOB	VARBINARY(MAX)	No usado: los archivos se referencian por URL.
JSON (semi estructurado)	JSONB	JSON	NVARCHAR(MAX) + CHECK (ISJSON(...) = 1)	PostgreSQL y MySQL guardan JSON binario validado; SQL Server 2019/2022 lo guarda como texto validado con ISJSON.

Las diferencias que más afectaron el diseño fueron tres. Primero, la fecha y hora: TIMESTAMP en PostgreSQL es una fecha, pero en MySQL solo llega hasta el año 2038 y en SQL Server TIMESTAMP ni siquiera es una fecha (es un contador de versión de fila), por eso se usó DATETIME y DATETIME2(0). Segundo, el texto: SQL Server necesita NVARCHAR para guardar tildes y eñes en Unicode y ya no recomienda TEXT. Tercero, los documentos JSON: solo

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

PostgreSQL tiene un tipo binario indexable (JSONB); MySQL tiene JSON validado y SQL Server 2022 no tiene tipo JSON.

Tipos de Datos - Tabla Comparativa

#	Tipo de Dato Genérico	Descripción	En PostgreSQL Versión: 16			En MySQL Versión: 8.0			En MS SQL Server Versión: 2022			
			Tipo de Dato	Dominio	Tamaño BYTES	Tipo de Dato	Dominio	Tamaño BYTES	Tipo de Dato	Dominio	Tamaño BYTES	Observaciones
1	AUTOINCREMENTABLE	Entero generado automáticamente como identificador secuencial	SMALLINT / INTEGER / BIGINT GENERATED ALWAYS AS IDENTITY	1 .. 32.767 / 2.147.483.647 / 9,2×10^18	2 / 4 / 8	SMALLINT / INT / BIGINT AUTO_INCREMENT	1 .. 32.767 / 2.147.483.647 / 9,2×10^18	2 / 4 / 8	SMALLINT / INT / BIGINT IDENTITY(1,1)	1 .. 32.767 / 2.147.483.647 / 9,2×10^18	2 / 4 / 8	PK sustitutas de todas las entidades fuertes. SMALLINT en catálogos, INT en entidades, BIGINT en tablas de alto volumen.
2	UUID	Identificador único universal no secuencial	UUID	128 bits (36 caracteres en texto)	16	BINARY(16) o CHAR(36)	No tiene tipo UUID nativo; función UUID()	16 / 36	UNIQUEIDENTIFIER	128 bits; función NEWID()	16	No se usa en el modelo: las PK enteras (4 bytes) son suficientes (observación del docente en la revisión de la EA3).
3	ENTERO	Número sin decimales	SMALLINT / INTEGER / BIGINT	±32.767 / ±2.147 millones / ±9,2×10^18	2 / 4 / 8	SMALLINT / INT / BIGINT	±32.767 / ±2.147 millones / ±9,2×10^18	2 / 4 / 8	SMALLINT / INT / BIGINT	±32.767 / ±2.147 millones / ±9,2×10^18	2 / 4 / 8	puntuacion (SMALLINT), cantidad_disponible y cupo_maximo (INT). MySQL y SQL Server también tienen TINYINT (1 byte).
4	DECIMAL	Número exacto con parte fraccionaria	NUMERIC(p,s)	Hasta 131.072 dígitos antes del punto	Variable (= 8 para 12,2)	DECIMAL(p,s)	Hasta 65 dígitos	6 para (12,2)	DECIMAL(p,s)	Hasta 38 dígitos	9 para (12,2)	Dinero: precio_producto, valor_total, valor_entrada en NUMERIC/DECIMAL(12,2). Nunca FLOAT/REAL por el redondeo.
5	LÓGICO	Valor verdadero / falso	BOOLEAN	TRUE / FALSE / NULL	1	BOOLEAN (alias de TINYINT(1))	0 / 1 / NULL	1	BIT	0 / 1 / NULL	1 bit (8 por byte)	Todo atributo de dos valores: es_privado, fue_leido, asistio, es_nuevo, estado_rol.
6	TEXTO	Cadena larga de longitud variable	TEXT	Hasta 1 GB	Variable	TEXT	Hasta 65.535 bytes	Longitud + 2	NVARCHAR(MAX)	Hasta 2 GB	Variable	contenido_publicacion y cuerpo_mensaje. En SQL Server TEXT está obsoleto.
7	CARACTER	Cadena corta de longitud limitada	VARCHAR(n) / CHAR(n)	Hasta n caracteres	n + 1	VARCHAR(n) / CHAR(n) (utf8mb4)	Hasta n caracteres	Hasta 4n + 1	NVARCHAR(n) / NCHAR(n)	Hasta n caracteres Unicode	2n + 2	Estados cortos con 3 o más valores (estado_usuario, estado_compra). Se prefiere VARCHAR a CHAR para no rellenar con espacios.
8	FECHA	Fecha sin hora	DATE	4713 a.C. .. 5874897 d.C.	4	DATE	1000-01-01 .. 9999-12-31	3	DATE	0001-01-01 .. 9999-12-31	3	fecha_nacimiento, fecha_creacion.
9	FECHA_HORA	Fecha y hora	TIMESTAMP	4713 a.C. .. 294276 d.C. (µs)	8	DATETIME	1000-01-01 .. 9999-12-31	5	DATETIME2(0)	0001-01-01 .. 9999-12-31	6	Trazabilidad (fecha_publicacion, fecha_compra). En MySQL TIMESTAMP llega solo a 2038; en SQL Server TIMESTAMP es rowversion, no una fecha.
10	HORA	Hora del día sin fecha	TIME	00:00:00 .. 24:00:00	8	TIME	-838:59:59 .. 838:59:59	3	TIME(0)	00:00:00 .. 23:59:59	3	No se usa: las fechas de eventos guardan fecha y hora juntas.
11	URL	Dirección web	VARCHAR(255)	Hasta 255 caracteres	Hasta 256	VARCHAR(255)	Hasta 255 caracteres	Hasta 1.021	NVARCHAR(255)	Hasta 255 caracteres	Hasta 512	url_multimedia, url_fotografia.
12	ENUM	Conjunto cerrado de valores	VARCHAR(n) + CHECK (IN ...)	Valores listados en el CHECK	Variable	VARCHAR(n) + CHECK (existe ENUM(...))	Valores listados en el CHECK	Variable	NVARCHAR(n) + CHECK (IN ...)	Valores listados en el CHECK	Variable	Se usa VARCHAR + CHECK en los tres para que la regla sea idéntica y fácil de ampliar.
13	TEXTO_CORTO	Cadena breve de tamaño limitado	VARCHAR(n)	Hasta n caracteres	n + 1	VARCHAR(n)	Hasta n caracteres	Hasta 4n + 1	NVARCHAR(n)	Hasta n caracteres	2n + 2	Nombres y títulos: nombre_grupo VARCHAR(80), asunto_mensaje VARCHAR(150).
14	BINARIO	Datos binarios	BYTEA	Hasta 1 GB	Variable	VARBINARY(n) / LONGBLOB	Hasta 4 GB	Variable	VARBINARY(MAX)	Hasta 2 GB	Variable	No se usa: los archivos se referencian por URL.
15	JSON	Documento semi estructurado (texto)	JSON	Texto JSON validado, se re-analiza en cada consulta	Variable	JSON	Binario validado (no existe JSONB)	Variable	NVARCHAR(MAX) + CHECK (ISJSON(...) = 1)	Texto validado con ISJSON	Variable	En SQL Server 2019/2022 no existe tipo JSON nativo (llega en SQL Server 2025).
16	JSONB	Documento semi estructurado (binario)	JSONB	Binario validado; índices GIN; operadores ->, ->>, ?, @>	Variable	JSON (equivalente)	Binario validado; índices sobre columnas generadas	Variable	NVARCHAR(MAX) + ISJSON	Funciones JSON_VALUE, OPENJSON	Variable	perfil.atributos_perfil y, en el script semi estructurado, usuario.perfil_extendido, evento.ubicacion_geo y registro_actividad.datos_actividad.
17	IMAGEN	Imagen	VARCHAR(255) (URL) / BYTEA	Ruta o binario	Variable	VARCHAR(255) (URL) / LONGBLOB	Ruta o binario	Variable	NVARCHAR(255) (URL) / VARBINARY(MAX)	Ruta o binario	Variable	url_fotografia guarda la URL; la imagen va en almacenamiento externo.
18	AUDIO	Audio	VARCHAR(255) (URL) / BYTEA	Ruta o binario	Variable	VARCHAR(255) (URL) / LONGBLOB	Ruta o binario	Variable	NVARCHAR(255) (URL) / VARBINARY(MAX)	Ruta o binario	Variable	No se usa en esta fase.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

El inventario es el mismo para los tres SGBD: 23 tablas con los mismos nombres en snake_case y en singular. Es el inventario corregido de la EA3, que ya había llevado el modelo lógico a 3FN, convertido las relaciones 1:N en claves foráneas y dejado tablas propias solo para las relaciones N:M y las entidades débiles. En la pestaña "Invetario de Tablas" de la hoja de cálculo se escriben una sola vez el nombre y la descripción de cada tabla; los tres diccionarios físicos toman esos valores por referencia de celda.

#	Tabla	Descripción	Tipo
1	rol	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).	Entidad
2	tipo_usuario	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).	Entidad
3	usuario	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.	Entidad
4	cuenta	Credenciales y estado de acceso del usuario, separadas de sus datos personales.	Entidad débil (1:1)
5	grupo	Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.	Entidad
6	perfil	Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.	Entidad
7	grupo_usuario	Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.	Relación N:M
8	usuario_sigue_usuario	Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).	Relación N:M recursiva
9	publicacion	Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo.	Entidad
10	comentario	Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.	Entidad
11	reaccion	Reacción breve (me gusta, me encanta...) de un usuario a una publicación; una sola por usuario y publicación.	Entidad débil
12	mensaje	Mensaje privado enviado por un usuario a otro dentro de la plataforma.	Entidad
13	notificacion	Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.	Entidad
14	tipo_servicio	Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC...).	Entidad
15	servicio	Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.	Entidad
16	solicitud_servicio	Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.	Relación N:M con atributos (entidad asociativa)
17	calificacion	Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.	Entidad débil
18	tipo_producto	Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets...).	Entidad
19	producto	Artículo que un usuario publica para la venta, con precio, existencias y condición.	Entidad
20	compra_producto	Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.	Relación N:M con atributos (entidad asociativa)
21	tipo_evento	Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto...).	Entidad
22	evento	Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.	Entidad
23	participacion_evento	Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.	Relación N:M con atributos

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Correcciones aplicadas desde la EA3

Además de las correcciones de la EA3, para esta tarea se aplicaron las observaciones de la revisión del docente:

- **Se eliminó uuid_usuario de la tabla usuario.** En la revisión de la EA3 el docente observó que el UUID no era necesario: id_usuario (INTEGER IDENTITY, 4 bytes) ya identifica al usuario y el UUID agregaba 16 bytes por fila, un índice único más y una columna que ninguna FK usaba.
- **condicion_producto pasó a es_nuevo BOOLEAN.** Era un VARCHAR con solo dos valores posibles (nuevo / usado). Un BOOLEAN ocupa 1 byte y no necesita CHECK. Los estados con tres o más valores (estado_solicitud, estado_cuenta, estado_membresia, entre otros) se mantienen en VARCHAR con CHECK, porque un BOOLEAN no puede representarlos.
- **CHECK de fecha_nacimiento con fecha fija.** La regla quedó como fecha_nacimiento >= '1900-01-01'. Una regla con CURRENT_DATE no es determinística y MySQL la rechaza dentro de un CHECK; con una fecha fija la misma regla funciona igual en los tres motores.
- **clave_cifrada pasó a VARCHAR(60).** Es el tamaño exacto del hash bcrypt que se guarda; no se reservan caracteres que nunca se usan.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Modelo Lógico - Tablas resultantes del Diccionario Genérico (corregido)

INVENTARIO DE TABLAS

Este inventario es el mismo para los 3 SGBDs: PostgreSQL, MySQL y MS SQL Server. Todos los diccionarios deben tomar la misma cantidad y los mismos nombres de esta tablas

#	Tabla	Descripción Tabla	Observaciones
1	rol	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).	Entidad. Catálogo independiente. Se elimina el FK a usuario para evitar referencia circular.
2	tipo_usuario	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).	Entidad. Catálogo independiente. Se elimina id_usuario_registra (referencia circular con usuario).
3	usuario	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.	Entidad. Absorbe las relaciones N:1 'tiene rol' y 'tiene tipo' como FK. Se eliminan usuario_tiene_rol y usuario_tiene_tipo. EA4: se elimina uuid_usuario (la PK entera basta); CHECK fecha_nacimiento >= 1900-01-01 (determinístico, válido en los 3 motores).
4	cuenta	Credenciales y estado de acceso del usuario, separadas de sus datos personales.	Entidad débil (1:1). PK = FK a usuario. Se elimina usuario_tiene_cuenta (duplicaba la relación 1:1). EA4: clave_cifrada pasa a VARCHAR(60), el tamaño exacto de un hash bcrypt.
5	grupo	Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.	Entidad. Se agregan es_privado y estado_grupo del modelo conceptual.
6	perfil	Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.	Entidad. Absorbe cuenta_tiene_perfil y grupo_define_perfil. CHECK de exclusividad usuario/grupo.
7	grupo_usuario	Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.	Relación N:M. Absorbe usuario_solicita_grupo y usuario_administra_grupo (estado_membresia y rol_en_grupo).
8	usuario_sigue_usuario	Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).	Relación N:M recursiva. Nueva exigencia de la EA3: conectar y seguir compañeros.
9	publicacion	Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo.	Entidad. Absorbe usuario_realiza_publicacion (era la misma tabla duplicada).
10	comentario	Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.	Entidad. Absorbe publicacion_tiene_comentario y usuario_comenta.
11	reaccion	Reacción breve (me gusta, me encanta...) de un usuario a una publicación; una sola por usuario y publicación.	Entidad débil. PK compuesta (id_publicacion, id_usuario). Absorbe publicacion_tiene_reaccion y usuario_reacciona.
12	mensaje	Mensaje privado enviado por un usuario a otro dentro de la plataforma.	Entidad. Absorbe usuario_envia_mensaje y usuario_recibe_mensaje.
13	notificacion	Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.	Entidad. Absorbe usuario_recibe_notificacion.
14	tipo_servicio	Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC...).	Entidad. Absorbe usuario_registra_tipo_servicio como FK id_usuario_registra.
15	servicio	Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.	Entidad. Absorbe usuario_ofrece_servicio y tipo_servicio_tiene_servicio como FK.
16	solicitud_servicio	Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.	Relación N:M con atributos (entidad asociativa). Implementa la relación 'solicita' y la traza de ventas de servicios exigida por la EA3.
17	calificacion	Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.	Entidad débil. Se liga a la solicitud para que solo califique quien contrató. Absorbe servicio_recibe_calificacion y usuario_califica.
18	tipo_producto	Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets...).	Entidad. Absorbe usuario_registra_tipo_producto como FK id_usuario_registra.
19	producto	Artículo que un usuario publica para la venta, con precio, existencias y condición.	Entidad. Absorbe usuario_vende_producto y tipo_producto_tiene_producto como FK. EA4: condicion_producto (nuevo/usado) se reemplaza por es_nuevo BOOLEAN.
20	compra_producto	Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.	Relación N:M con atributos (entidad asociativa). Implementa la relación 'compra' y la traza de ventas de productos exigida por la EA3.
21	tipo_evento	Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto...).	Entidad. Absorbe usuario_registra_tipo_evento como FK id_usuario_registra.
22	evento	Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.	Entidad. Absorbe usuario_organiza_evento y tipo_evento_tiene_evento como FK.
23	participacion_evento	Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.	Relación N:M con atributos. Implementa la relación 'participa' y la traza de ventas de eventos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3. Diccionario de Datos y Scripts de PostgreSQL

3.1- Diccionario de Datos Físico (SGBD PostgreSQL)

El Diccionario de Datos Físico de PostgreSQL está en la pestaña "Diccionario Datos - PostgreSQL". Cada tabla toma su número, nombre y descripción del inventario por referencia de celda (='Invetario de Tablas'!B4, etc.), de modo que un cambio de nombre se refleja en los tres diccionarios. Para cada campo se registra la descripción, el tipo en PostgreSQL, el tamaño, si admite nulos, si es PK, FK o UK, y una nota con el valor por defecto, la acción referencial y las restricciones CHECK. El diccionario se generó a partir del catálogo de la base ya creada (information_schema), por lo que coincide con el script de creación.

Justificación de los tipos de datos

- **VARCHAR(n) y TEXT:** VARCHAR con tamaño cuando el negocio define un máximo (nombre_rol 30, correo_electronico 120). El límite valida y documenta el dato. TEXT solo para contenido_publicacion y cuerpo_mensaje, que no deben cortarse.
- **numero_documento y telefono_usuario como VARCHAR:** son identificadores, no cantidades: pueden tener ceros a la izquierda, letras (pasaporte) o el signo +.
- **SMALLINT, INTEGER o BIGINT:** SMALLINT (2 bytes) en catálogos y puntuacion; INTEGER (4 bytes) en usuario, grupo, servicio, producto y evento; BIGINT (8 bytes) en las tablas que crecen con cada interacción: publicacion, comentario, mensaje, notificacion, solicitud_servicio y compra_producto. Cada FK tiene el mismo tipo que la PK que referencia.
- **NUMERIC(12,2) para dinero:** exacto, sin errores de redondeo; admite valores hasta 9.999.999.999,99 pesos.
- **TIMESTAMP y DATE:** toda acción guarda fecha y hora con DEFAULT CURRENT_TIMESTAMP para ordenar y auditar; DATE solo donde la hora no aporta (fecha_nacimiento, fecha_creacion del grupo).
- **BOOLEAN:** todo atributo de dos valores: estado de los catálogos, es_privado, fue_leido, asistio, esta_disponible, es_nuevo.
- **JSONB:** atributos_perfil, porque intereses y hobbies cambian de un usuario a otro.

Justificación de las claves primarias

- **Entidades fuertes:** PK sustituta numérica GENERATED ALWAYS AS IDENTITY: corta, estable y no cambia si el usuario corrige sus datos. IDENTITY es el estándar SQL y no permite insertar valores manuales por error.
- **Entidades débiles:** cuenta usa la PK del usuario (1:1) y calificacion la de la solicitud; reaccion usa la PK compuesta (id_publicacion, id_usuario), que impide reaccionar dos veces.
- **Relaciones N:M:** grupo_usuario, usuario_sigue_usuario y participacion_evento usan las dos FK como PK compuesta, lo que impide duplicar la membresía, el seguimiento o la inscripción.

Tablas implementadas y su tipo en cada SGBD

La siguiente relación muestra cada tabla con el tipo de cada campo en los tres SGBD. Las columnas Nulo y Clave son iguales en los tres motores; la única diferencia de claves está en perfil, donde SQL Server implementa la unicidad de id_usuario e id_grupo con un índice único filtrado (sección 5.2).

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

1. rol — Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_rol	SMALLINT	SMALLINT	SMALLINT	No	PK
nombre_rol	VARCHAR(30)	VARCHAR(30)	NVARCHAR(30)	No	UK
descripcion_rol	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
estado_rol	BOOLEAN	BOOLEAN	BIT	No	

2. tipo_usuario — Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_tipo_usuario	SMALLINT	SMALLINT	SMALLINT	No	PK
nombre_tipo_usuario	VARCHAR(40)	VARCHAR(40)	NVARCHAR(40)	No	UK
descripcion_tipo_usuario	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
estado_tipo_usuario	BOOLEAN	BOOLEAN	BIT	No	

3. usuario — Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_usuario	INTEGER	INT	INT	No	PK
id_tipo_usuario	SMALLINT	SMALLINT	SMALLINT	No	FK
id_rol	SMALLINT	SMALLINT	SMALLINT	No	FK
numero_documento	VARCHAR(20)	VARCHAR(20)	NVARCHAR(20)	No	UK
nombres_usuario	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	
apellidos_usuario	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	
correo_electronico	VARCHAR(120)	VARCHAR(120)	NVARCHAR(120)	No	UK
telefono_usuario	VARCHAR(15)	VARCHAR(15)	NVARCHAR(15)	Sí	
fecha_nacimiento	DATE	DATE	DATE	Sí	
fecha_registro	TIMESTAMP	DATETIME	DATETIME2(0)	No	
estado_usuario	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	

4. cuenta — Credenciales y estado de acceso del usuario, separadas de sus datos personales.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_usuario	INTEGER	INT	INT	No	PK/FK
nombre_cuenta	VARCHAR(30)	VARCHAR(30)	NVARCHAR(30)	No	UK
clave_cifrada	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	
estado_cuenta	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	
fecha_ultimo_acceso	TIMESTAMP	DATETIME	DATETIME2(0)	Sí	
intentos_fallidos	SMALLINT	SMALLINT	SMALLINT	No	

5. grupo — Comunidad de usuarios reunida por un interés común: grupos de estudio, semilleros, hackatones, clubes.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_grupo	INTEGER	INT	INT	No	PK
nombre_grupo	VARCHAR(80)	VARCHAR(80)	NVARCHAR(80)	No	UK
descripcion_grupo	VARCHAR(500)	VARCHAR(500)	NVARCHAR(500)	Sí	
fecha_creacion	DATE	DATE	DATE	No	
es_privado	BOOLEAN	BOOLEAN	BIT	No	

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
estado_grupo	VARCHAR(10)	VARCHAR(10)	NVARCHAR(10)	No	

6. perfil — Información pública e intereses de un usuario o de un grupo; incluye atributos semiestructurados JSONB.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_perfil	INTEGER	INT	INT	No	PK
id_usuario	INTEGER	INT	INT	Sí	FK/UK
id_grupo	INTEGER	INT	INT	Sí	FK/UK
titular_perfil	VARCHAR(100)	VARCHAR(100)	NVARCHAR(100)	No	
area_estudio	VARCHAR(100)	VARCHAR(100)	NVARCHAR(100)	Sí	
descripcion_perfil	VARCHAR(500)	VARCHAR(500)	NVARCHAR(500)	Sí	
atributos_perfil	JSONB	JSON	NVARCHAR(MAX)	Sí	
url_fotografia	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
fecha_actualizacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	

7. grupo_usuario — Membresía de usuarios en grupos (N:M), con estado de solicitud y rol dentro del grupo.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_grupo	INTEGER	INT	INT	No	PK/FK
id_usuario	INTEGER	INT	INT	No	PK/FK
fecha_ingreso	TIMESTAMP	DATETIME	DATETIME2(0)	No	
estado_membresia	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	
rol_en_grupo	VARCHAR(15)	VARCHAR(15)	NVARCHAR(15)	No	

8. usuario_sigue_usuario — Relación recursiva de seguimiento entre usuarios (seguidor -> seguido).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_usuario_seguidor	INTEGER	INT	INT	No	PK/FK
id_usuario_seguido	INTEGER	INT	INT	No	PK/FK
fecha_seguimiento	TIMESTAMP	DATETIME	DATETIME2(0)	No	

9. publicacion — Contenido que un usuario comparte en la Red, opcionalmente dentro de un grupo.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_publicacion	BIGINT	BIGINT	BIGINT	No	PK
id_usuario	INTEGER	INT	INT	No	FK
id_grupo	INTEGER	INT	INT	Sí	FK
titulo_publicacion	VARCHAR(150)	VARCHAR(150)	NVARCHAR(150)	Sí	
contenido_publicacion	TEXT	TEXT	NVARCHAR(MAX)	No	
url_multimedia	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
fecha_publicacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	
estado_publicacion	VARCHAR(10)	VARCHAR(10)	NVARCHAR(10)	No	

10. comentario — Respuesta que un usuario escribe sobre una publicación; el autor puede ser distinto al de la publicación.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_comentario	BIGINT	BIGINT	BIGINT	No	PK
id_publicacion	BIGINT	BIGINT	BIGINT	No	FK
id_usuario	INTEGER	INT	INT	No	FK

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
texto_comentario	VARCHAR(1000)	VARCHAR(1000)	NVARCHAR(1000)	No	
fecha_comentario	TIMESTAMP	DATETIME	DATETIME2(0)	No	
estado_comentario	VARCHAR(10)	VARCHAR(10)	NVARCHAR(10)	No	

11. reaccion — Reacción breve (me gusta, me encanta...) de un usuario a una publicación; una sola por usuario y publicación.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_publicacion	BIGINT	BIGINT	BIGINT	No	PK/FK
id_usuario	INTEGER	INT	INT	No	PK/FK
tipo_reaccion	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	
fecha_reaccion	TIMESTAMP	DATETIME	DATETIME2(0)	No	

12. mensaje — Mensaje privado enviado por un usuario a otro dentro de la plataforma.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_mensaje	BIGINT	BIGINT	BIGINT	No	PK
id_usuario_remitente	INTEGER	INT	INT	No	FK
id_usuario_destinatario	INTEGER	INT	INT	No	FK
asunto_mensaje	VARCHAR(150)	VARCHAR(150)	NVARCHAR(150)	Sí	
cuerpo_mensaje	TEXT	TEXT	NVARCHAR(MAX)	No	
fecha_envio	TIMESTAMP	DATETIME	DATETIME2(0)	No	
fue_leido	BOOLEAN	BOOLEAN	BIT	No	

13. notificacion — Aviso generado por el sistema y dirigido a un usuario sobre la actividad que le concierne.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_notificacion	BIGINT	BIGINT	BIGINT	No	PK
id_usuario	INTEGER	INT	INT	No	FK
tipo_notificacion	VARCHAR(15)	VARCHAR(15)	NVARCHAR(15)	No	
referencia_origen	BIGINT	BIGINT	BIGINT	Sí	
texto_notificacion	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	No	
fecha_generacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	
fue_leida	BOOLEAN	BOOLEAN	BIT	No	

14. tipo_servicio — Catálogo de categorías de servicio definido por la administración (asesoría, tutoría, reparación de PC...).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_tipo_servicio	SMALLINT	SMALLINT	SMALLINT	No	PK
id_usuario_registra	INTEGER	INT	INT	No	FK
nombre_tipo_servicio	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	UK
descripcion_tipo_servicio	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
estado_tipo_servicio	BOOLEAN	BOOLEAN	BIT	No	

15. servicio — Servicio concreto que un usuario ofrece a la comunidad, con precio, duración y ubicación.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_servicio	INTEGER	INT	INT	No	PK
id_usuario	INTEGER	INT	INT	No	FK
id_tipo_servicio	SMALLINT	SMALLINT	SMALLINT	No	FK

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
nombre_servicio	VARCHAR(100)	VARCHAR(100)	NVARCHAR(100)	No	
descripcion_servicio	VARCHAR(1000)	VARCHAR(1000)	NVARCHAR(1000)	Sí	
precio_servicio	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	
duracion_minutos	INTEGER	INT	INT	Sí	
ubicacion_servicio	VARCHAR(150)	VARCHAR(150)	NVARCHAR(150)	Sí	
esta_disponible	BOOLEAN	BOOLEAN	BIT	No	
fecha_publicacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	

16. solicitud_servicio — Traza de cada contratación de un servicio por parte de otro usuario, con estado y valor acordado.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_solicitud	BIGINT	BIGINT	BIGINT	No	PK
id_servicio	INTEGER	INT	INT	No	FK
id_usuario_solicitante	INTEGER	INT	INT	No	FK
fecha_solicitud	TIMESTAMP	DATETIME	DATETIME2(0)	No	
estado_solicitud	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	
valor_acordado	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	

17. calificacion — Valoración (1 a 5) y reseña que deja quien contrató un servicio, una por solicitud.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_solicitud	BIGINT	BIGINT	BIGINT	No	PK/FK
puntuacion	SMALLINT	SMALLINT	SMALLINT	No	
texto_calificacion	VARCHAR(500)	VARCHAR(500)	NVARCHAR(500)	Sí	
fecha_calificacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	

18. tipo_producto — Catálogo de categorías de producto definido por la administración (papelería, libros, ropa, tickets...).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_tipo_producto	SMALLINT	SMALLINT	SMALLINT	No	PK
id_usuario_registra	INTEGER	INT	INT	No	FK
nombre_tipo_producto	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	UK
descripcion_tipo_producto	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
estado_tipo_producto	BOOLEAN	BOOLEAN	BIT	No	

19. producto — Artículo que un usuario publica para la venta, con precio, existencias y condición.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_producto	INTEGER	INT	INT	No	PK
id_usuario	INTEGER	INT	INT	No	FK
id_tipo_producto	SMALLINT	SMALLINT	SMALLINT	No	FK
nombre_producto	VARCHAR(100)	VARCHAR(100)	NVARCHAR(100)	No	
descripcion_producto	VARCHAR(1000)	VARCHAR(1000)	NVARCHAR(1000)	Sí	
precio_producto	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	
cantidad_disponible	INTEGER	INT	INT	No	
es_nuevo	BOOLEAN	BOOLEAN	BIT	No	
fecha_publicacion	TIMESTAMP	DATETIME	DATETIME2(0)	No	

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
estado_producto	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	

20. compra_producto — Traza de cada venta de un producto: comprador, cantidad, precio unitario y total.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_compra	BIGINT	BIGINT	BIGINT	No	PK
id_producto	INTEGER	INT	INT	No	FK
id_usuario_comprador	INTEGER	INT	INT	No	FK
fecha_compra	TIMESTAMP	DATETIME	DATETIME2(0)	No	
cantidad	INTEGER	INT	INT	No	
precio_unitario	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	
valor_total	NUMERIC(14,2)	DECIMAL(14,2)	DECIMAL(14,2)	No	
estado_compra	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	

21. tipo_evento — Catálogo de categorías de evento definido por la administración (congreso, taller, feria, concierto...).

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_tipo_evento	SMALLINT	SMALLINT	SMALLINT	No	PK
id_usuario_registra	INTEGER	INT	INT	No	FK
nombre_tipo_evento	VARCHAR(60)	VARCHAR(60)	NVARCHAR(60)	No	UK
descripcion_tipo_evento	VARCHAR(255)	VARCHAR(255)	NVARCHAR(255)	Sí	
estado_tipo_evento	BOOLEAN	BOOLEAN	BIT	No	

22. evento — Actividad académica o social organizada por un usuario, con fechas, lugar, cupo y valor de entrada.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_evento	INTEGER	INT	INT	No	PK
id_usuario_organizador	INTEGER	INT	INT	No	FK
id_tipo_evento	SMALLINT	SMALLINT	SMALLINT	No	FK
nombre_evento	VARCHAR(120)	VARCHAR(120)	NVARCHAR(120)	No	
descripcion_evento	VARCHAR(1000)	VARCHAR(1000)	NVARCHAR(1000)	Sí	
fecha_inicio	TIMESTAMP	DATETIME	DATETIME2(0)	No	
fecha_fin	TIMESTAMP	DATETIME	DATETIME2(0)	No	
lugar_evento	VARCHAR(150)	VARCHAR(150)	NVARCHAR(150)	No	
cupo_maximo	INTEGER	INT	INT	Sí	
valor_entrada	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	
estado_evento	VARCHAR(12)	VARCHAR(12)	NVARCHAR(12)	No	

23. participacion_evento — Inscripción de usuarios a eventos: traza de pago, asistencia, me gusta y comentario posterior.

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
id_evento	INTEGER	INT	INT	No	PK/FK
id_usuario	INTEGER	INT	INT	No	PK/FK
fecha_inscripcion	TIMESTAMP	DATETIME	DATETIME2(0)	No	
valor_pagado	NUMERIC(12,2)	DECIMAL(12,2)	DECIMAL(12,2)	No	
asistio	BOOLEAN	BOOLEAN	BIT	No	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Campo	PostgreSQL	MySQL	SQL Server	Nulo	Clave
me_gusta	BOOLEAN	BOOLEAN	BIT	No	
comentario_evento	VARCHAR(500)	VARCHAR(500)	NVARCHAR(500)	Sí	
fecha_hora_comentario	TIMESTAMP	DATETIME	DATETIME2(0)	Sí	

PostgreSQL - Tablas en Detalle

1	Tabla	rol	Descripción Tabla	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	SMALLINT	2 bytes	No	S			GENERATED ALWAYS AS IDENTITY
2	nombre_rol	Nombre del rol: administrador, auxiliar, miembro, visitante	VARCHAR	30	No			S	
3	descripcion_rol	Alcance de los permisos que otorga el rol	VARCHAR	255	Sí				
4	estado_rol	Indica si el rol puede asignarse (TRUE = vigente)	BOOLEAN	1 byte	No				DEFAULT true

2	Tabla	tipo_usuario	Descripción Tabla	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_tipo_usuario	Identificador único del tipo de usuario	SMALLINT	2 bytes	No	S			GENERATED ALWAYS AS IDENTITY
2	nombre_tipo_usuario	Nombre del tipo: estudiante, docente, egresado...	VARCHAR	40	No			S	
3	descripcion_tipo_usuario	Aclaración del alcance del tipo de usuario	VARCHAR	255	Sí				
4	estado_tipo_usuario	Indica si el tipo está vigente para nuevas asignaciones	BOOLEAN	1 byte	No				DEFAULT true

3	Tabla	usuario	Descripción Tabla	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador interno del usuario	INTEGER	4 bytes	No	S			GENERATED ALWAYS AS IDENTITY
2	id_tipo_usuario	Tipo de usuario (vínculo con la Institución)	SMALLINT	2 bytes	No		S		Ref. tipo_usuario(id_tipo_usuario) ON DELETE RESTRICT ON UPDATE CASCADE
3	id_rol	Rol de permisos en la Red	SMALLINT	2 bytes	No		S		Ref. rol(id_rol) ON DELETE RESTRICT ON UPDATE CASCADE
4	numero_documento	Documento de identidad	VARCHAR	20	No			S	
5	nombres_usuario	Nombres de la persona	VARCHAR	60	No				
6	apellidos_usuario	Apellidos de la persona	VARCHAR	60	No				
7	correo_electronico	Correo de contacto, no repetible	VARCHAR	120	No			S	CHECK (correo_electronico LIKE '%_@_%._%')
8	telefono_usuario	Teléfono de contacto	VARCHAR	15	Sí				
9	fecha_nacimiento	Fecha de nacimiento (análisis demográfico)	DATE	4 bytes	Sí				CHECK (fecha_nacimiento >= '1900-01-01')
10	fecha_registro	Fecha y hora de registro en la Red	TIMESTAMP	8 bytes	No				DEFAULT CURRENT_TIMESTAMP
11	estado_usuario	Estado: activo, inactivo o suspendido	VARCHAR	12	No				DEFAULT 'activo'; CHECK (estado_usuario IN ('activo', 'inactivo', 'suspendido'))

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.2.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos

El script 20262-bd1-ea4-E-E-scripts-01-postgre-crea.sql contiene las 23 sentencias CREATE TABLE numeradas igual que el inventario. Parte del script de la EA3 con las correcciones de la sección 2. Se ejecuta completo en pgAdmin4 (Query Tool) sobre una base vacía y crea 23 tablas, 34 claves foráneas y las restricciones CHECK sin errores. Las tablas se crean en siete bloques, primero las independientes y después las dependientes, de modo que toda tabla referenciada por una FK ya existe cuando se crea la tabla que la usa.

Cada CREATE TABLE sigue la misma estructura: columnas con tipo, tamaño, NOT NULL y DEFAULT; CONSTRAINT pk_ con la clave primaria; CONSTRAINT uk_ para valores únicos; CONSTRAINT fk_ con REFERENCES, ON DELETE y ON UPDATE; y CONSTRAINT ck_ con las reglas de negocio. Nombrar las restricciones permite reconocerlas en los mensajes de error y modificarlas con ALTER TABLE. La misma estructura y el mismo orden se conservan en los scripts de MySQL y SQL Server.

Bloque	Tablas (en orden de ejecución)	Por qué en ese orden
1	rol, tipo_usuario	Catálogos sin FK.
2	usuario, cuenta	usuario depende de los catálogos; cuenta de usuario.
3	grupo, perfil, grupo_usuario, usuario_sigue_usuario	Dependen de usuario y grupo.
4	publicacion, comentario, reaccion, mensaje, notificacion	Contenido creado por usuarios.
5	tipo_servicio, servicio, solicitud_servicio, calificacion	Catálogo → oferta → contratación → calificación.
6	tipo_producto, producto, compra_producto	Catálogo → producto → venta.
7	tipo_evento, evento, participacion_evento	Catálogo → evento → inscripción.

Claves foráneas y acciones referenciales en los tres SGBD

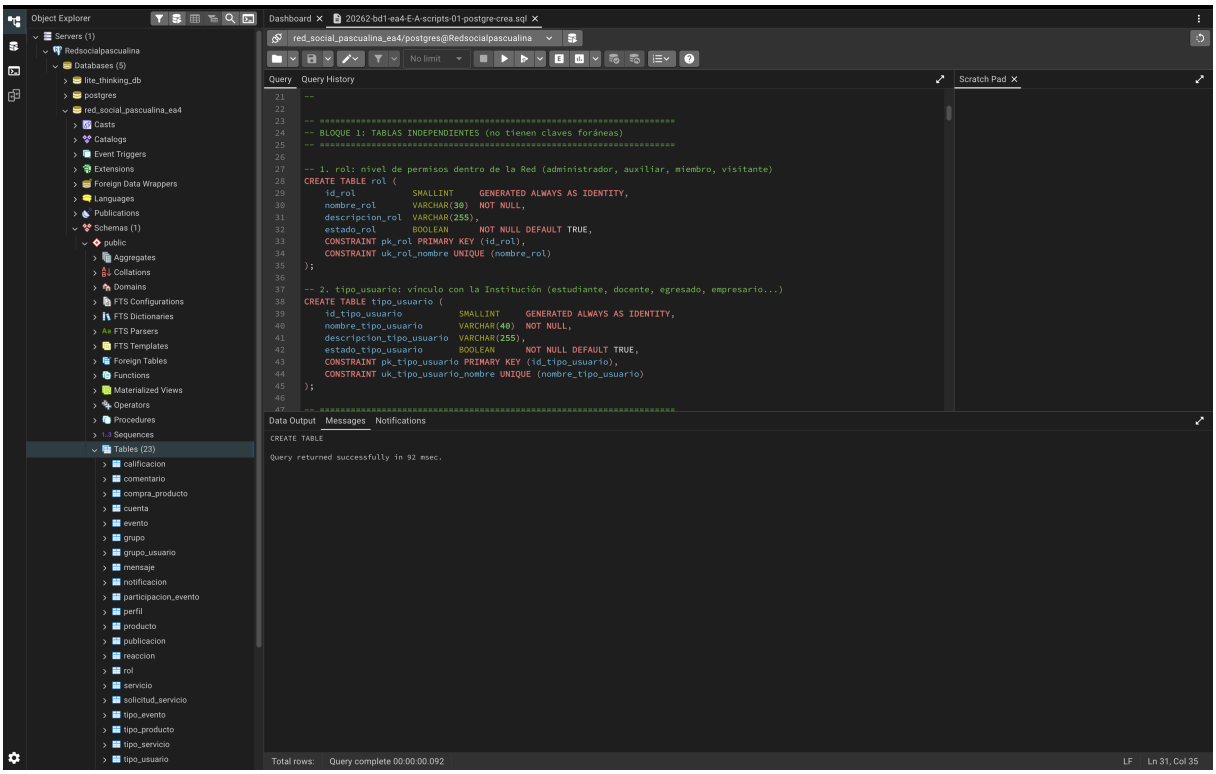
Las acciones referenciales se decidieron en PostgreSQL según el tipo de dependencia: CASCADE para los datos que no tienen sentido sin su padre (cuenta, perfil, membresías, reacciones, notificaciones), RESTRICT para catálogos y para todo lo que es autoría o traza (publicaciones, mensajes, ventas, contrataciones), y SET NULL solo en publicacion.id_grupo. Los usuarios no se borran: se desactivan con estado_usuario (borrado lógico). La tabla muestra cómo quedó cada FK en los tres motores; las diferencias se explican en las secciones 4.2 y 5.2.

Tabla hija	Campo FK	Tabla padre	PostgreSQL (DEL / UPD)	MySQL (DEL / UPD)	SQL Server (DEL / UPD)
usuario	id_tipo_usuario	tipo_usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
usuario	id_rol	rol	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
cuenta	id_usuario	usuario	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
perfil	id_usuario	usuario	CASC. / CASC.	RESTR. / RESTR.	CASC. / NO ACT.
perfil	id_grupo	grupo	CASC. / CASC.	RESTR. / RESTR.	CASC. / NO ACT.
grupo_usuario	id_grupo	grupo	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
grupo_usuario	id_usuario	usuario	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
usuario_sigue_usuario	id_usuario_seguidor	usuario	CASC. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
usuario_sigue_usuario	id_usuario_seguido	usuario	CASC. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
publicacion	id_usuario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
publicacion	id_grupo	grupo	SET NULL / CASC.	SET NULL / RESTR.	SET NULL / NO ACT.
comentario	id_publicacion	publicacion	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
comentario	id_usuario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
reaccion	id_publicacion	publicacion	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
reaccion	id_usuario	usuario	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
mensaje	id_usuario_remitente	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
mensaje	id_usuario_destinatario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
notificacion	id_usuario	usuario	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
tipo_servicio	id_usuario_registra	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
servicio	id_usuario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
servicio	id_tipo_servicio	tipo_servicio	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
solicitud_servicio	id_servicio	servicio	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
solicitud_servicio	id_usuario_solicitante	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
calificacion	id_solicitud	solicitud_servicio	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
tipo_producto	id_usuario_registra	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tabla hija	Campo FK	Tabla padre	PostgreSQL (DEL / UPD)	MySQL (DEL / UPD)	SQL Server (DEL / UPD)
producto	id_usuario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
producto	id_tipo_producto	tipo_producto	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
compra_producto	id_producto	producto	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
compra_producto	id_usuario_comprador	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
tipo_evento	id_usuario_registra	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
evento	id_usuario_organizador	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
evento	id_tipo_evento	tipo_evento	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.
participacion_evento	id_evento	evento	CASC. / CASC.	CASC. / RESTR.	CASC. / NO ACT.
participacion_evento	id_usuario	usuario	RESTR. / CASC.	RESTR. / RESTR.	NO ACT. / NO ACT.

RESTR. = RESTRICT, NO ACT. = NO ACTION, CASC. = CASCADE.

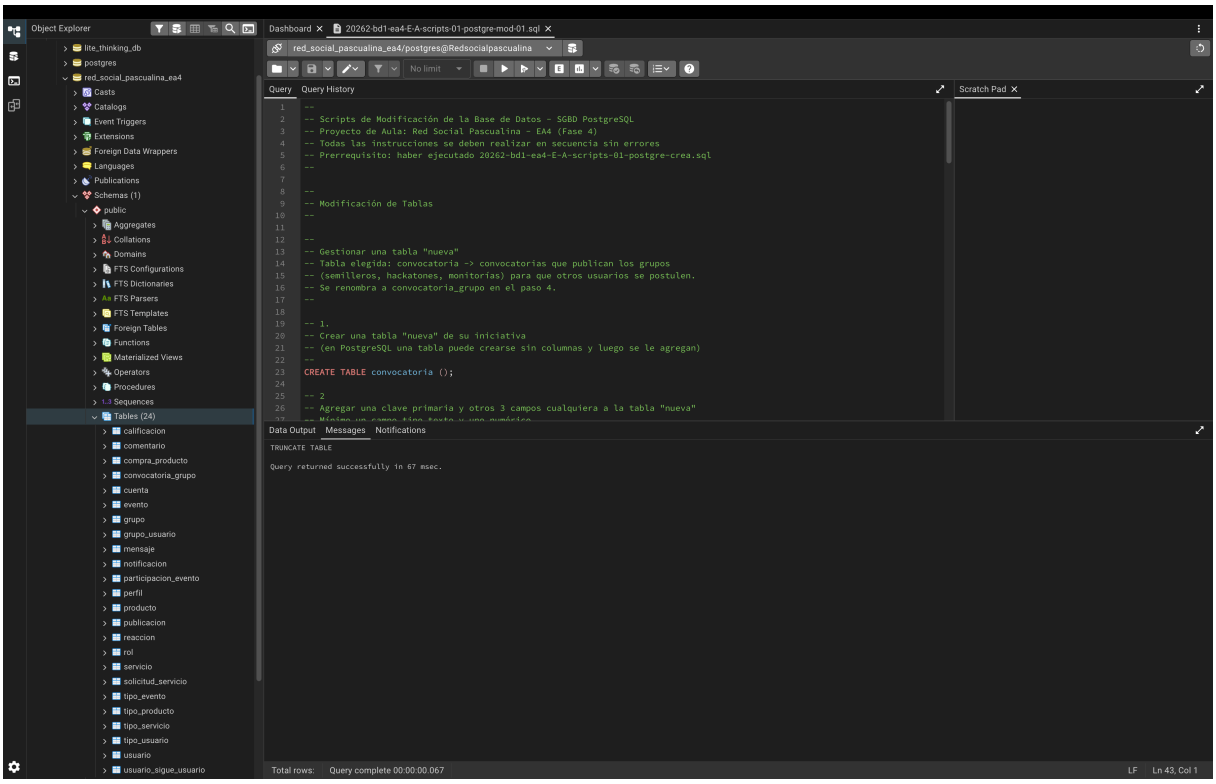


Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.3..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos ESTRUCTURADOS

El script 20262-bd1-ea4-E-E-scripts-01-postgre-mod-01.sql cumple los 12 puntos de la plantilla en orden y se ejecuta de una sola vez después del script de creación. La tabla nueva es convocatoria: las convocatorias que publican los grupos (semilleros, hackatones, monitorías) para que otros usuarios se postulen. En el paso 4 se renombra a convocatoria_grupo y todos los pasos siguientes trabajan sobre la tabla renombrada.

Paso	Instrucción	Qué hace
1	CREATE TABLE convocatoria ();	Crea la tabla sin columnas (PostgreSQL lo permite).
2	ADD COLUMN id_convocatoria INTEGER GENERATED ALWAYS AS IDENTITY + PRIMARY KEY; ADD COLUMN titulo VARCHAR(100), descripcion TEXT, numero_plazas INTEGER	Clave primaria y tres campos (dos de texto y uno numérico).
3	DROP COLUMN descripcion_convocatoria	Quita uno de los campos.
4	RENAME TO convocatoria_grupo; RENAME CONSTRAINT pk_convocatoria TO pk_convocatoria_grupo	Cambia el nombre de la tabla y, por consistencia, el de su PK.
5	ADD COLUMN codigo_convocatoria VARCHAR(12) NOT NULL + UNIQUE	Campo único.
6	ADD COLUMN fecha_inicio, fecha_fin DATE + CHECK (fecha_fin >= fecha_inicio)	Dos fechas con control de orden.
7	ADD COLUMN numero_postulados INTEGER DEFAULT 0 + CHECK (>= 0)	Entero que no puede ser negativo.
8	ALTER COLUMN titulo_convocatoria TYPE VARCHAR(150)	Amplía el campo de texto de 100 a 150.
9	ALTER COLUMN numero_plazas TYPE SMALLINT, SET NOT NULL + CHECK (BETWEEN 1 AND 100)	Cambia el campo numérico y le pone control de rango.
10	CREATE INDEX ix_convocatoria_grupo_fecha_inicio	Índice sobre fecha_inicio, el campo por el que se consultan.
11	DROP CONSTRAINT ck_convocatoria_grupo_fechas; DROP COLUMN fecha_fin	Elimina una de las fechas (primero la restricción que la usa).
12	INSERT de 2 filas + SELECT; TRUNCATE TABLE convocatoria_grupo RESTART IDENTITY	Borra todos los datos sin traza fila por fila y reinicia el contador de la PK.



Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.4..- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos para datos SEMI ESTRUCTURADOS

El script 20262-bd1-ea4-E-E-scripts-01-postgre-mod-02.sql agrega datos semi estructurados con JSONB. Se ejecuta después de los dos scripts anteriores. JSONB guarda el documento en formato binario ya validado, permite índices GIN y consultas por clave (->, ->>, ?, @>); JSON, en cambio, guarda el texto tal cual y lo vuelve a analizar en cada consulta. Por eso se eligió JSONB.

- **A. usuario.perfil_extendido (propuesta de la plantilla).** No todos los usuarios tienen la misma información: un estudiante tiene intereses, habilidades y materias; un docente, líneas de investigación y publicaciones; un empresario, empresa, sector y ofertas. Con una columna por cada dato la tabla quedaría llena de NULL. La columna se agrega como JSONB NOT NULL DEFAULT '{}', con un CHECK que obliga a que sea un objeto y un índice GIN. Se diferencia de perfil.atributos_perfil, que es la información pública que el usuario muestra en su perfil; perfil_extendido guarda los datos institucionales que dependen del tipo de usuario.
- **B. evento.ubicacion_geo (otro dato semi estructurado en una tabla existente).** Guarda las coordenadas del lugar del evento, por ejemplo {"lat": 6.2795, "lon": -75.5850, "sede": "Robledo"}. Un CHECK valida que lat y lon sean números dentro de rango, aunque el resto del documento sea libre.
- **C. Tabla nueva registro_actividad (otro dato semi estructurado en una tabla nueva).** Bitácora de alto volumen de lo que pasa en la red: inicios de sesión desde la app, vistas de publicaciones y lecturas de dispositivos IoT (por ejemplo, el lector de carné en la entrada de un evento). Cada origen envía datos distintos, por eso el detalle va en datos_actividad JSONB. La PK es BIGINT porque la tabla puede llegar a millones de filas; origen es un SMALLINT con CHECK (1 web, 2 móvil, 3 IoT) y la FK a usuario usa ON DELETE SET NULL para no perder la estadística si se elimina un usuario.
- **D y E. Datos de ejemplo y consultas.** Se insertan los tres usuarios del Anexo B (estudiante, docente y empresario), un evento con ubicación y tres registros de actividad, y se ejecutan seis consultas sobre JSONB.

Fragmento del script (parte A):

```
ALTER TABLE usuario
    ADD COLUMN perfil_extendido JSONB NOT NULL DEFAULT '{}'::JSONB;
ALTER TABLE usuario
    ADD CONSTRAINT ck_usuario_perfil_extendido_objeto
    CHECK (jsonb_typeof(perfil_extendido) = 'object');
CREATE INDEX ix_usuario_perfil_extendido
    ON usuario USING GIN (perfil_extendido);
```

Consultas ejecutadas y resultado obtenido

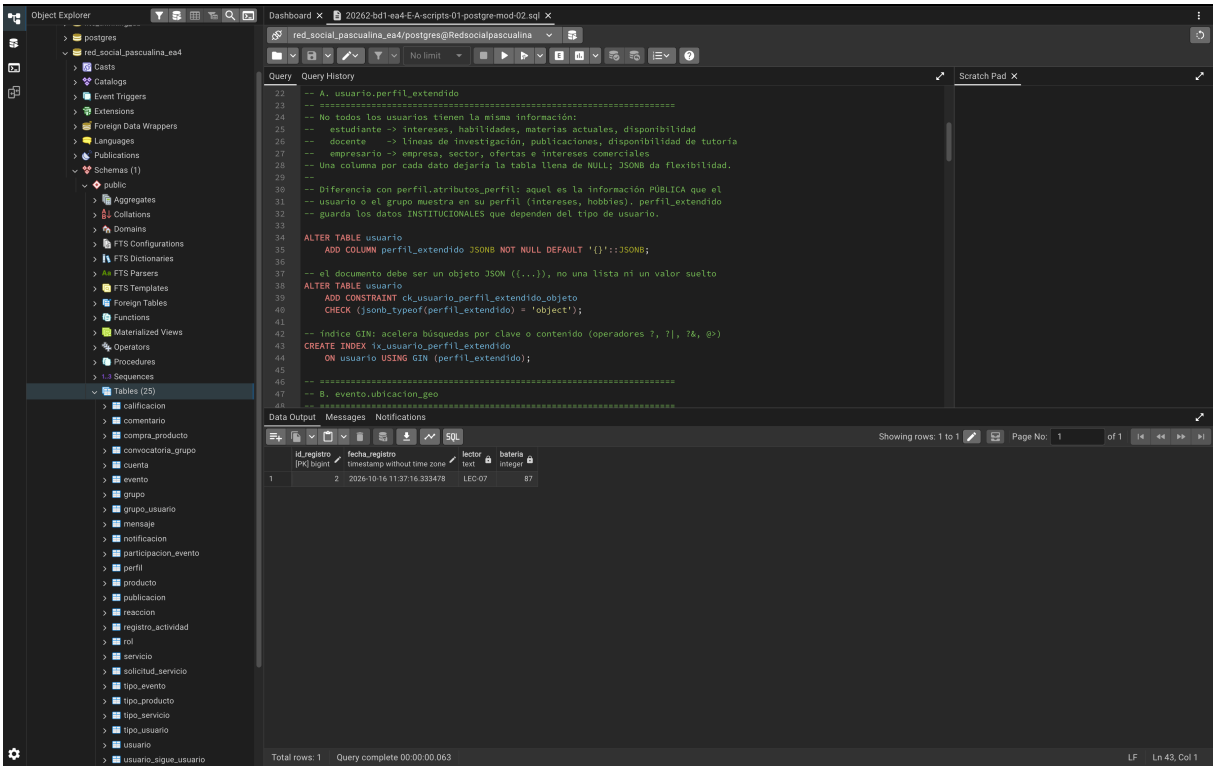
Consulta	Condición	Resultado
Usuarios por tipo	JOIN con tipo_usuario WHERE nombre_tipo_usuario = 'estudiante'	Ana Torres
Interesados en IA	perfil_extendido -> 'intereses' ? 'IA'	Ana Torres
Habilidades	perfil_extendido -> 'habilidades'	Python intermedio, SQL básico
Docentes en IoT	perfil_extendido @> '{"lineas_investigacion": ["IoT"]}'	Carlos Gómez
Coordenadas	(ubicacion_geo ->> 'lat')::NUMERIC	Taller de JSONB: 6.2795, -75.5850, Robledo
Lecturas IoT	origen = 3 AND (datos_actividad ->> 'bateria')::INTEGER < 90	Lector LEC-07, batería 87

Justificación para Big Data e IoT

Un dispositivo IoT o una aplicación móvil no siempre envía la misma estructura: un lector de carné manda el código del lector, la posición y el nivel de batería; la app manda el sistema operativo y la versión; el navegador

manda el tiempo de lectura. Si cada dato fuera una columna, cada dispositivo nuevo obligaría a un ALTER TABLE sobre una tabla con millones de filas. Con JSONB el esquema relacional se mantiene fijo (quién, cuándo, de dónde) y la parte variable entra en el documento.

Esto no significa abandonar el modelo relacional. Los datos que se cruzan con otras tablas o que tienen reglas de negocio (usuario, tipo, fecha, origen) siguen siendo columnas con tipo, FK y CHECK; solo lo que varía va en JSONB. Los índices GIN permiten buscar dentro del documento sin recorrer toda la tabla, y BIGINT y SMALLINT mantienen pequeñas las filas de una tabla que crece sin parar. Es el mismo criterio que usan las plataformas que reciben datos de sensores: estructura fija donde se puede, flexibilidad donde se necesita.



4. Diccionario de Datos y Scripts de MySQL

4.1- Diccionario de Datos Físico (SGBD MySql)

El Diccionario de Datos Físico de MySQL está en la pestaña "Diccionario Datos - MySQL" y tiene la misma estructura que el de PostgreSQL: número, nombre y descripción de cada tabla tomados por referencia del inventario, y todas las tablas en detalle. Cambian el tipo de dato, el tamaño en bytes y las notas (AUTO_INCREMENT en lugar de IDENTITY y las acciones referenciales de MySQL). Los tipos se verificaron contra information_schema de la base creada en MySQL 8.0: los 152 campos coinciden.

MySQL - Tablas en Detalle

1	Tabla	rol	Descripción Tabla	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	SMALLINT	2 bytes	No	S			AUTO_INCREMENT
2	nombre_rol	Nombre del rol: administrador, auxiliar, miembro, visitante	VARCHAR	30	No			S	
3	descripcion_rol	Alcance de los permisos que otorga el rol	VARCHAR	255	Sí				
4	estado_rol	Indica si el rol puede asignarse (TRUE = vigente)	BOOLEAN	1 byte (TINYINT(1))	No				DEFAULT TRUE

2	Tabla	tipo_usuario	Descripción Tabla	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_tipo_usuario	Identificador único del tipo de usuario	SMALLINT	2 bytes	No	S			AUTO_INCREMENT
2	nombre_tipo_usuario	Nombre del tipo: estudiante, docente, egresado...	VARCHAR	40	No			S	
3	descripcion_tipo_usuario	Aclaración del alcance del tipo de usuario	VARCHAR	255	Sí				
4	estado_tipo_usuario	Indica si el tipo está vigente para nuevas asignaciones	BOOLEAN	1 byte (TINYINT(1))	No				DEFAULT TRUE

3	Tabla	usuario	Descripción Tabla	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.				Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MySQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador interno del usuario	INT	4 bytes	No	S			AUTO_INCREMENT
2	id_tipo_usuario	Tipo de usuario (vínculo con la Institución)	SMALLINT	2 bytes	No		S		Ref. tipo_usuario(id_tipo_usuario) ON DELETE RESTRICT ON UPDATE RESTRICT
3	id_rol	Rol de permisos en la Red	SMALLINT	2 bytes	No		S		Ref. rol(id_rol) ON DELETE RESTRICT ON UPDATE RESTRICT
4	numero_documento	Documento de identidad	VARCHAR	20	No			S	
5	nombres_usuario	Nombres de la persona	VARCHAR	60	No				
6	apellidos_usuario	Apellidos de la persona	VARCHAR	60	No				
7	correo_electronico	Correo de contacto, no repetible	VARCHAR	120	No			S	CHECK (correo_electronico LIKE '%_@_%._%')
8	telefono_usuario	Teléfono de contacto	VARCHAR	15	Sí				
9	fecha_nacimiento	Fecha de nacimiento (análisis demográfico)	DATE	3 bytes	Sí				CHECK (fecha_nacimiento >= '1900-01-01')
10	fecha_registro	Fecha y hora de registro en la Red	DATETIME	5 bytes	No				DEFAULT CURRENT_TIMESTAMP
11	estado_usuario	Estado: activo, inactivo o suspendido	VARCHAR	12	No				DEFAULT 'activo'; CHECK (estado_usuario IN ('activo', 'inactivo', 'suspendido'))

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

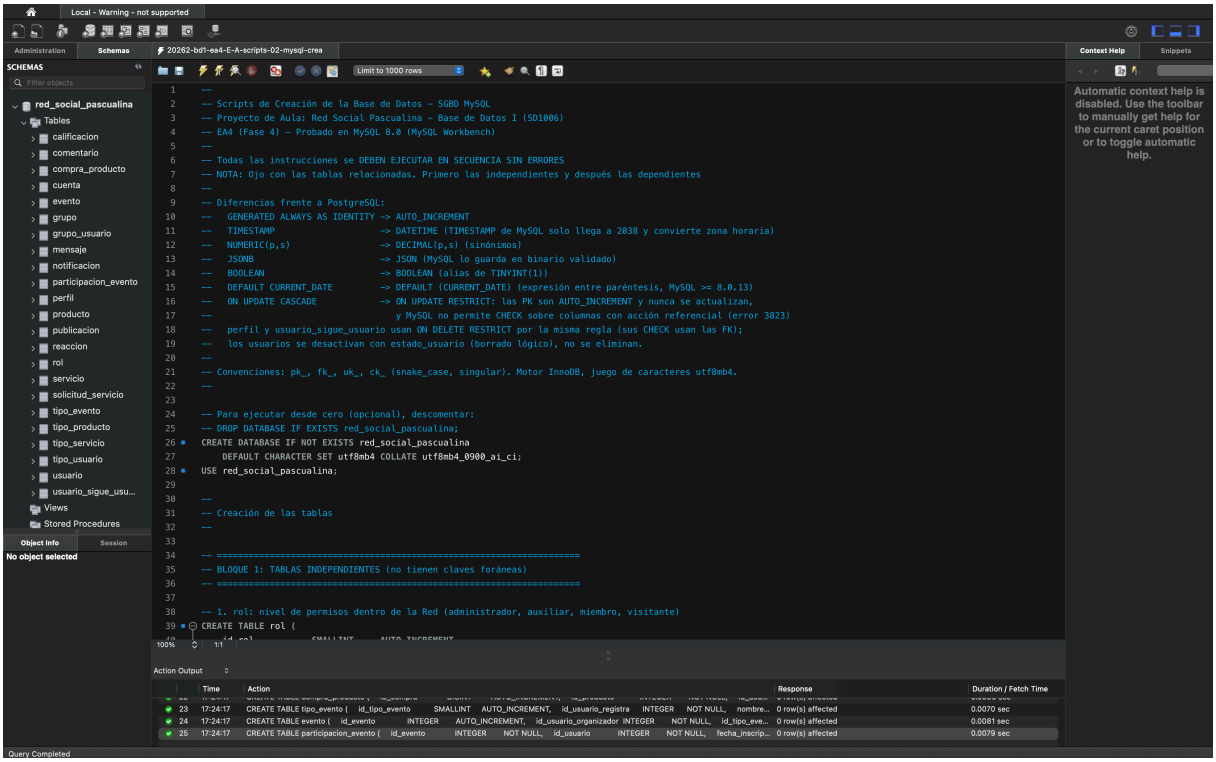
4.2.- MYSQL. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos

El script 20262-bd1-ea4-E-E-scripts-02-mysql-crea.sql crea la base red_social_pascualina con utf8mb4 y las 23 tablas en el mismo orden que PostgreSQL. Se ejecuta completo en MySQL Workbench; en la prueba sobre MySQL 8.0 creó 23 tablas, 34 claves foráneas y 33 restricciones CHECK sin errores.

PostgreSQL	MySQL	Motivo
GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	MySQL no implementa IDENTITY del estándar.
TIMESTAMP	DATETIME	TIMESTAMP de MySQL solo llega a 2038 y convierte la zona horaria.
NUMERIC(12,2)	DECIMAL(12,2)	Sinónimos.
JSONB	JSON	MySQL guarda JSON binario validado; no existe JSONB.
BOOLEAN	BOOLEAN (= TINYINT(1))	Se guarda como 0 / 1.
DEFAULT CURRENT_DATE	DEFAULT (CURRENT_DATE)	Una expresión como default va entre paréntesis (MySQL 8.0.13+).
ON UPDATE CASCADE	ON UPDATE RESTRICT	Las PK son AUTO_INCREMENT y nunca se actualizan; además evita el error 3823.
ON DELETE CASCADE en perfil y usuario_sigue_usuario	ON DELETE RESTRICT	Error 3823: MySQL no permite un CHECK sobre una columna FK con acción referencial.
(tabla)	ENGINE=InnoDB, utf8mb4	InnoDB es el motor con FK y transacciones; utf8mb4 guarda tildes, ñ y emojis.

La diferencia más importante es el error 3823: MySQL no deja declarar un CHECK sobre una columna que también es FK con acción CASCADE o SET NULL. En perfil (CHECK de exclusividad usuario/grupo) y usuario_sigue_usuario (CHECK que impide seguirse a sí mismo) se cambió ON DELETE CASCADE por RESTRICT, y en todas las tablas ON UPDATE CASCADE por RESTRICT. Ninguna regla de negocio se pierde: las PK nunca se actualizan y los usuarios no se borran, se desactivan con estado_usuario.

Después de crear la base se comprobó que las reglas funcionan: MySQL rechazó un usuario que se sigue a sí mismo, un perfil sin titular, una fecha de nacimiento de 1850 y el borrado de un rol en uso; y aceptó el DEFAULT (CURRENT_DATE) de grupo y las consultas JSON_EXTRACT sobre atributos_perfil.



Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5. Diccionario de Datos y Scripts de MS SQL Server

5.1- Diccionario de Datos Físico (SGBD MS SQL Server)

El Diccionario de Datos Físico de SQL Server está en la pestaña "Diccionario Datos - MS SQL Serv", con la misma estructura y las mismas referencias al inventario. Los tipos cambian más que en MySQL: NVARCHAR en lugar de VARCHAR, BIT en lugar de BOOLEAN, DATETIME2(0) en lugar de TIMESTAMP y NVARCHAR(MAX) para TEXT y JSONB. En perfil, id_usuario e id_grupo se marcan como UK con un índice único filtrado.

MS SQL Server - Tablas en Detalle

1	Tabla	rol	Descripción Tabla	Catálogo de niveles de permiso con que un usuario opera en la Red (administrador, auxiliar, miembro, visitante).			Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_rol	Identificador único del rol	SMALLINT	2 bytes	No	S			IDENTITY(1,1)
2	nombre_rol	Nombre del rol: administrador, auxiliar, miembro, visitante	NVARCHAR	30	No			S	
3	descripcion_rol	Alcance de los permisos que otorga el rol	NVARCHAR	255	Sí				
4	estado_rol	Indica si el rol puede asignarse (TRUE = vigente)	BIT	1 bit	No				DEFAULT 1

2	Tabla	tipo_usuario	Descripción Tabla	Catálogo del vínculo del usuario con la Institución (estudiante, docente, egresado, empleado, empresario).			Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_tipo_usuario	Identificador único del tipo de usuario	SMALLINT	2 bytes	No	S			IDENTITY(1,1)
2	nombre_tipo_usuario	Nombre del tipo: estudiante, docente, egresado...	NVARCHAR	40	No			S	
3	descripcion_tipo_usuario	Aclaración del alcance del tipo de usuario	NVARCHAR	255	Sí				
4	estado_tipo_usuario	Indica si el tipo está vigente para nuevas asignaciones	BIT	1 bit	No				DEFAULT 1

3	Tabla	usuario	Descripción Tabla	Persona registrada en la Red Social Pascualina; eje de toda la actividad de la plataforma.			Versión	4.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador interno del usuario	INT	4 bytes	No	S			IDENTITY(1,1)
2	id_tipo_usuario	Tipo de usuario (vínculo con la Institución)	SMALLINT	2 bytes	No		S		Ref. tipo_usuario(id_tipo_usuario) ON DELETE NO ACTION ON UPDATE NO ACTION
3	id_rol	Rol de permisos en la Red	SMALLINT	2 bytes	No		S		Ref. rol(id_rol) ON DELETE NO ACTION ON UPDATE NO ACTION
4	numero_documento	Documento de identidad	NVARCHAR	20	No			S	
5	nombres_usuario	Nombres de la persona	NVARCHAR	60	No				
6	apellidos_usuario	Apellidos de la persona	NVARCHAR	60	No				
7	correo_electronico	Correo de contacto, no repetible	NVARCHAR	120	No			S	CHECK (correo_electronico LIKE '%_@_%._%')
8	telefono_usuario	Teléfono de contacto	NVARCHAR	15	Sí				
9	fecha_nacimiento	Fecha de nacimiento (análisis demográfico)	DATE	3 bytes	Sí				CHECK (fecha_nacimiento >= '1900-01-01')
10	fecha_registro	Fecha y hora de registro en la Red	DATETIME2(0)	6 bytes	No				DEFAULT SYSDATETIME()
11	estado_usuario	Estado: activo, inactivo o suspendido	NVARCHAR	12	No				DEFAULT 'activo'; CHECK (estado_usuario IN ('activo', 'inactivo', 'suspendido'))

5.2.- SQL Server. Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos

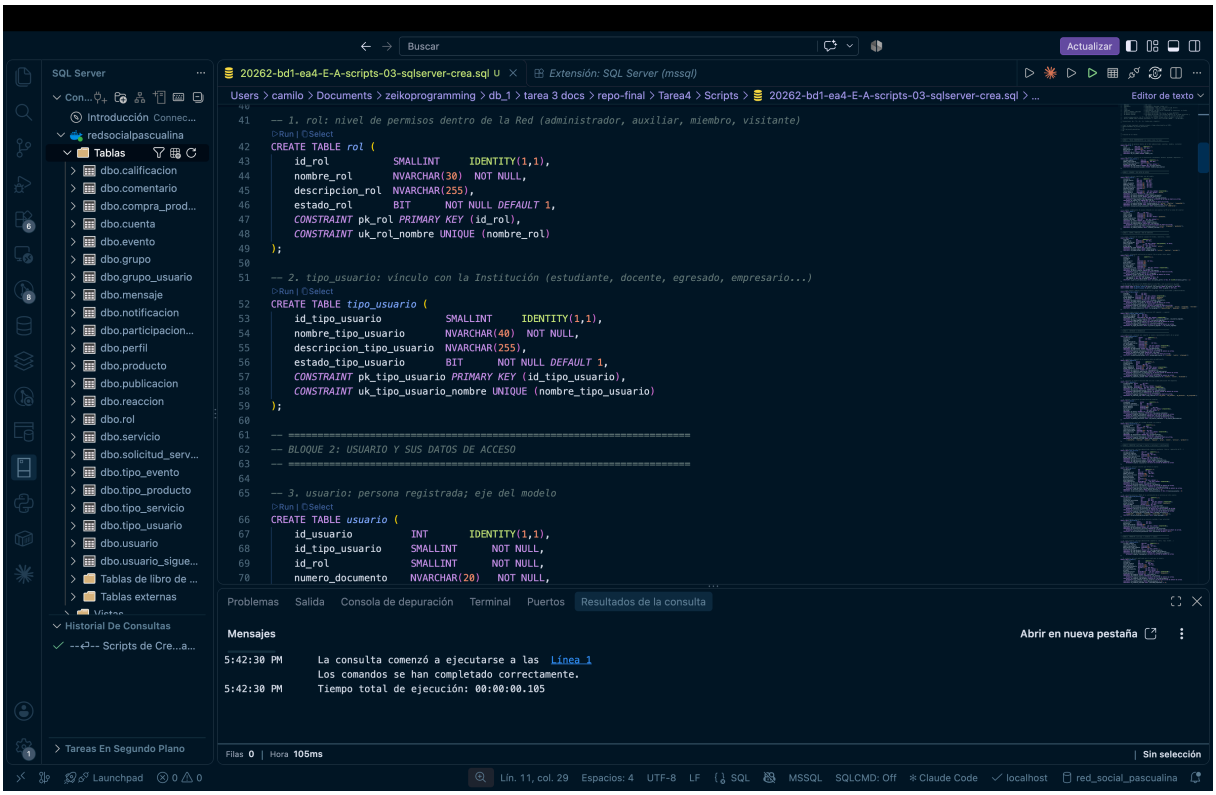
El script 20262-bd1-ea4-E-E-scripts-03-sqlserver-crea.sql crea las 23 tablas en el mismo orden. Al inicio trae comentadas las instrucciones CREATE DATABASE y USE con GO, porque en SQL Server Management Studio la base se crea y se selecciona antes de ejecutar el resto.

PostgreSQL	SQL Server	Motivo
GENERATED ALWAYS AS IDENTITY	IDENTITY(1,1)	Sintaxis propia de SQL Server.
VARCHAR(n)	NVARCHAR(n)	Unicode: tildes y ñ sin depender de la intercalación.
TEXT	NVARCHAR(MAX)	TEXT está obsoleto en SQL Server.
BOOLEAN	BIT	Valores 0 / 1.
TIMESTAMP	DATETIME2(0)	En SQL Server TIMESTAMP es rowversion, no una fecha.
DEFAULT CURRENT_TIMESTAMP / CURRENT_DATE	SYSDATETIME() / CAST(GETDATE() AS DATE)	Funciones de fecha de T-SQL.
JSONB	NVARCHAR(MAX) + CHECK (ISJSON(...) = 1)	SQL Server 2022 no tiene tipo JSON; se valida con ISJSON.
ON DELETE RESTRICT	ON DELETE NO ACTION	SQL Server no tiene RESTRICT; NO ACTION tiene el mismo efecto.
ON UPDATE CASCADE	ON UPDATE NO ACTION	Una columna IDENTITY no se puede actualizar.
ON DELETE CASCADE en usuario_sigue_usuario	NO ACTION	Error 1785: dos FK a usuario con CASCADE forman rutas múltiples de cascada.
UNIQUE (id_usuario), UNIQUE (id_grupo) en perfil	CREATE UNIQUE INDEX ... WHERE ... IS NOT NULL	En SQL Server UNIQUE admite un solo NULL; el índice filtrado ignora los NULL.

Dos reglas de SQL Server obligaron a cambios. La primera es el error 1785: SQL Server rechaza una FK con CASCADE si, desde una tabla, existen dos caminos de borrado en cascada hacia la misma tabla. usuario_sigue_usuario tiene dos FK a usuario (seguidor y seguido), así que ambas quedan en NO ACTION. Se revisaron las 34 FK y no quedan rutas múltiples. La segunda es que una restricción UNIQUE en SQL Server solo admite un NULL; como en perfil cada fila tiene id_usuario o id_grupo y el otro en NULL, la unicidad se implementa con un índice único filtrado (WHERE id_usuario IS NOT NULL), que solo indexa los valores informados.

El script se ejecutó completo sobre SQL Server 2022 (16.0, CU26) en un contenedor Docker, conectado desde Visual Studio Code con la extensión SQL Server (mssql), porque SQL Server Management Studio solo existe para Windows. Terminó sin errores con 23 tablas, 34 claves foráneas, 34 restricciones CHECK (las 33 del modelo más la de ISJSON sobre atributos_perfil) y los dos índices únicos filtrados de perfil. También se comprobó que las reglas se cumplen: un usuario que se sigue a sí mismo lo rechaza ck_sigue_distintos y un atributos_perfil que no es JSON lo rechaza ck_perfil_atributos_json.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)



6.- Análisis de resultados (Grupo)

Implementar el mismo modelo en tres SGBD mostró que el modelo lógico es independiente del motor, pero el modelo físico no. Las 23 tablas, las 34 claves foráneas y las reglas de negocio son las mismas en PostgreSQL, MySQL y SQL Server, y sin embargo ninguno de los tres scripts es igual a otro. Partir de un modelo ya corregido en la EA3 fue lo que hizo posible la traducción: el inventario, los nombres y las relaciones no cambiaron, solo la forma de escribirlos en cada motor.

En los tipos de datos, la mayor parte de la traducción fue directa: los enteros, DECIMAL, DATE y VARCHAR existen en los tres. Las diferencias aparecieron en los tipos que parecen iguales y no lo son. TIMESTAMP es una fecha en PostgreSQL, una fecha limitada a 2038 en MySQL y un contador de versión en SQL Server; si se hubiera copiado el nombre sin revisar la documentación, la base de SQL Server habría quedado con columnas de fecha que no guardan fechas. Lo mismo pasa con BOOLEAN, que en MySQL es un entero y en SQL Server se llama BIT, y con el texto, que en SQL Server necesita NVARCHAR para guardar tildes y eñes. Los datos semi estructurados son el caso más distinto: solo PostgreSQL tiene un tipo binario indexable (JSONB).

En las relaciones, la consistencia se mantuvo con decisiones distintas en cada motor. PostgreSQL acepta cualquier combinación de acciones referenciales. MySQL no permite un CHECK sobre una columna FK con acción CASCADE (error 3823) y SQL Server no permite rutas múltiples de cascada (error 1785). En ambos casos la solución fue usar RESTRICT o NO ACTION, que es coherente con el diseño: las claves primarias son autoincrementales y nunca se actualizan, y los usuarios se desactivan por estado en lugar de borrarse, así que ninguna regla de negocio se perdió. La integridad también se comprobó con datos: en PostgreSQL y en MySQL se insertaron registros válidos e inválidos y el motor rechazó los inválidos.

Los scripts de modificación mostraron que una base de datos no se diseña una sola vez. ALTER TABLE permite agregar campos, cambiar tamaños, renombrar tablas y agregar restricciones sobre una base existente, pero el orden importa: para eliminar una fecha primero hay que eliminar el CHECK que la usa. Finalmente, el uso de JSONB demostró que el modelo relacional puede incorporar datos flexibles sin perder la estructura: lo que tiene reglas queda en columnas con tipo, FK y CHECK, y solo lo que varía entre usuarios o dispositivos va en el documento. El resultado es un modelo consistente en tres motores, con diccionarios generados de la misma fuente y scripts que se ejecutan de principio a fin sin errores.

7.- Reflexiones y conclusiones individuales

Juan Camilo González Hoyos

Esta tarea la realicé solo, y eso me obligó a entender cada parte del proceso en lugar de repartirla. Lo primero que aprendí es que un modelo bien hecho en un motor no se copia y pega en otro. Cuando pasé el script de PostgreSQL a MySQL, el script falló en tablas que en PostgreSQL funcionaban perfectamente, y el mensaje de error (3823) me llevó a descubrir que MySQL no permite combinar un CHECK con una clave foránea que tenga acción en cascada. En SQL Server pasó algo parecido con las rutas múltiples de cascada. Antes de esta tarea habría pensado que SQL es un solo lenguaje; ahora sé que el estándar es la base, pero cada motor tiene reglas propias que hay que leer en su documentación.

También entendí que los tipos de datos son parte del diseño y no un detalle de sintaxis. El caso de TIMESTAMP en SQL Server me pareció el más claro: tiene el mismo nombre que en PostgreSQL y significa algo completamente distinto. Las correcciones que hizo el docente sobre la EA3 fueron en la misma línea: quitar el UUID cuando una clave entera ya identifica al usuario y usar BOOLEAN cuando un estado solo tiene dos valores. Son decisiones

pequeñas por fila, pero en una red social con miles de usuarios y millones de interacciones definen cuánto ocupa la base y qué tan rápido responde.

La parte de datos semi estructurados fue la más nueva para mí. Siempre había visto el modelo relacional y los documentos JSON como opciones opuestas, y en esta tarea vi que se complementan: los datos con reglas se quedan en columnas y lo que cambia entre usuarios o dispositivos va en JSONB, con índices que permiten consultarlo. Eso se relaciona directamente con lo que hago desarrollando software, donde los datos de aplicaciones móviles y de dispositivos llegan con estructuras distintas y hay que decidir qué se valida en la base y qué se deja flexible.

Para mi vida profesional me llevo una forma de trabajar: generar los scripts y los diccionarios desde una misma fuente, probar cada script en el motor real antes de entregarlo e insertar datos inválidos a propósito para comprobar que las restricciones funcionan. También me llevo la importancia de versionar el trabajo en un repositorio, porque después de las fases anteriores entendí que un proyecto de datos necesita una sola versión de la verdad. Lo que sigue en el curso, las consultas y la manipulación de datos, va a construirse sobre esta base, y quiero que esa base esté bien hecha.

8.- Informe (calidad de presentación y respeto de los requerimientos)

Entregables de la tarea (carpeta Tarea4 del repositorio), con el nombre de la plantilla:

- Tarea4/Informe/20262-bd1-ea4-E-E-informe.pdf: este informe.
- Tarea4/Diccionarios/20262-bd1-ea4-E-E-resultados.xlsx: tipos de datos, inventario y los tres diccionarios físicos.
- Tarea4/Scripts/20262-bd1-ea4-E-E-scripts-01-postgre-crea.sql: creación en PostgreSQL.
- Tarea4/Scripts/20262-bd1-ea4-E-E-scripts-01-postgre-mod-01.sql: modificación de datos estructurados (12 pasos).
- Tarea4/Scripts/20262-bd1-ea4-E-E-scripts-01-postgre-mod-02.sql: modificación para datos semi estructurados (JSONB).
- Tarea4/Scripts/20262-bd1-ea4-E-E-scripts-02-mysql-crea.sql: creación en MySQL.
- Tarea4/Scripts/20262-bd1-ea4-E-E-scripts-03-sqlserver-crea.sql: creación en MS SQL Server.
- Tarea4/Video/README.md: enlace al video de sustentación.

9.- Video de Sustentación

Video "20262-bd1-ea4-E-E-video": presentación del estudiante y ejecución de los scripts de creación (PostgreSQL, MySQL y SQL Server) y de modificación (estructurados y semi estructurados) en PostgreSQL.

https://youtu.be/dxgV99aT1H8

10.- Repositorio GIT

Enlace al repositorio (carpeta Tarea4):

https://github.com/ZeikoDev/bd1-20262-g051-equipo-A/tree/main/Tarea4

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea		Peso	Cal
1	Tabla Comparativa de tipos de Datos		20	
2	Inventario de tablas (anterior tarea)		10	
3	3.1.- Diccionario de Datos Físico (PostgreSQL)		40	
	3.2.- Scripts de CREACIÓN de Bases de Datos en PostgreSQL		30	
	3.3.- Scripts MODIFICACIÓN BD-PostgreSQL (Estructurados)			
	3.4.- Scripts MODIFICACIÓN BD-PostgreSQL (Semi Estructurados)		20	
4	4.1.- Diccionario de Datos Físico (MySQL)		40	
	4.2.- Scripts de CREACIÓN de Bases de Datos en MySQL		30	
5	5.1.- Diccionario de Datos Físico (MS SQL Server)		40	
	5.2.- Scripts de CREACIÓN de Bases de Datos en (MS SQL Server)		30	
6	Análisis de resultados (en equipo - 300 palabras mínimo)		20	
7	Reflexiones y Conclusiones individuales (300 palabras mínimo)		30	
8	Informe: Estructura, Contenido y Calidad de presentación		40	
9	Video de Sustentación y Defensa “en vivo”		100	
10	Repositorio GIT		20	
	NOTA = xx/500 =	Total	500	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO A
Tipos de Datos Genéricos – Proyecto Red Apicola

Tipo de Dato Genérico	Descripción Conceptual	Uso Recomendado	Ejemplo en la Red Social
AUTOINCREMENTABLE	Número entero generado automáticamente como identificador único secuencial.	Claves primarias técnicas.	id_usuario, id_publicacion
UUID	Identificador único universal no secuencial.	Identificadores públicos o distribuidos.	uuid_usuario
ENTERO	Número sin decimales.	Contadores, edades, cantidades.	numero_likes, edad
DECIMAL	Número con parte fraccionaria.	Valores monetarios o métricas precisas.	costo_publicidad
LÓGICO	Valor booleano (verdadero/falso).	Estados binarios.	estado_activo, es_admin
TEXTO	Cadena de longitud variable amplia.	Descripciones, comentarios, publicaciones.	contenido_publicacion
CARACTER	Cadena corta de longitud fija o limitada.	Códigos, siglas, género, estado corto.	codigo_programa, genero
FECHA	Representa una fecha sin hora.	Cumpleaños, fecha_registro.	fecha_nacimiento
FECHA_HORA	Representa fecha y hora.	Trazabilidad de eventos.	fecha_creacion_post
IMAGEN	Archivo o referencia binaria de imagen.	Fotos de perfil o adjuntos gráficos.	foto_perfil
AUDIO	Archivo o referencia binaria de sonido.	Notas de voz.	audio_mensaje
URL	Dirección web válida.	Enlaces externos o multimedia.	enlace_video
ENUM	Conjunto limitado de valores predefinidos.	Estados controlados.	tipo_reaccion (LIKE, LOVE, WOW)
TEXTO_CORTO (opcional)	Cadena breve de tamaño limitado.	Títulos o nombres cortos.	titulo_grupo

BINARIO (opcional)	**Datos en formato binario.**	**Archivos adjuntos generales.**	**archivo_adjunto**

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO B

Ejemplo de Inserción/Consulta de Información No Relacional
Tipos de Datos Semi-Estructurados (JSON/JSONB)

```
- Datos para un estudiante
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Ana Torres',
    'ana.torres@pascual.edu.co',
    'estudiante',
    '{
        "intereses": ["programación", "videojuegos", "IA"],
        "habilidades": [
            {"nombre": "Python", "nivel": "intermedio"},
            {"nombre": "SQL", "nivel": "básico"}
        ],
        "materias_actuales": ["Bases de Datos I", "Algoritmos"],
        "disponibilidad": {
            "dias": ["lunes", "miércoles"],
            "horario": "tarde"
        }
    }'
);

- Datos para un docente
INSERT INTO usuarios (nombre, correo, tipo_usuario, perfil_extendido)
VALUES (
    'Carlos Gómez',
    'carlos.gomez@pascual.edu.co',
    'docente',
    '{
        "lineas_investigacion": ["Big Data", "IoT"],
        "publicaciones": [
            {"titulo": "Análisis de Datos", "anio": 2023},
            {"titulo": "IoT en educación", "anio": 2024}
        ],
        "disponibilidad_tutoria": {
            "dias": ["martes", "jueves"],
            "horario": "mañana"
        }
    }'
);

- Consultas
- Buscar usuarios por perfil
SELECT id_usuario,nombre,tipo_usuario FROM usuarios WHERE tipo_usuario= 'estudiante';

- Buscar usuarios interesados en "IA"
SELECT nombre, perfil_extendido FROM usuarios
WHERE perfil_extendido -> 'intereses' ? 'IA';

- Obtener habilidades de un usuario
SELECT nombre, perfil_extendido -> 'habilidades' AS habilidades FROM usuarios;
```